第 2 讲：浮点与条件分支

ICS 第一次小测复习 · 发布版 · 2026-10-13 · 独立答案另册

1. 二进制小数：小数点右边也有位权
2. 存下来的 exp、frac，不等于公式里的 E、M
3. 解码和编码都用固定步骤
4. 范围和精度：浮点数是一把间距会变的尺
5. 舍入：先找相邻刻度，再判断中点
6. 浮点加乘与 C 转换：每次运算都可能丢信息
7. 零、无穷、NaN 与“看位”
8. 条件码：CPU 记下的四个一位结果
9. 同一次 cmp，为什么有符号和无符号会走不同分支?
10. setcc 写一字节，返回整数还需整理高位
11. if 与 cmov：先说明执行了什么，再谈结果
12. 独立练习：先交过程，再核对答案
13. 过关检查与资料定位

本讲是复习计划的第二天，对应课程第 3 讲和第 5 讲的前半部分，并非课程第 2 讲。

今天要完成两件事：把一个浮点位串可靠地还原成数值，并且看懂一段汇编为什么走某条分支。前一件事训练你区分“存下来的字段”和“字段表示的数”；后一件事训练你区分“计算得到的位”与“如何解释这些位”。它们都依赖第一天的补码、无符号数、寄存器宽度和操作数顺序。

先用自己的话回答三个入口问题：`0xff` 按 8 位无符号数和补码各表示什么？`movq (%rdi), %rax` 取的是地址还是内容？向 `%eax` 写入会怎样影响 `%rax` 的高 32 位？若仍不确定，先回到第 1 讲修这几个点。本讲不要求你已经会函数调用栈。

阅读时先学到一个例题并自己重算，再继续下一段。文末必做题共 30 分，首次独立达到 24 分为当天检查门槛；答案另放在第 2 讲练习答案。不要边做边看答案。这里没有使用 2025 第一次小测的具体题目。

1. 二进制小数：小数点右边也有位权

十进制的 `0.375` 是三项位权的和；二进制也一样，只是小数点右边的位权依次是 $2^{-1}, 2^{-2}, 2^{-3}, \ldots$。因此

$$101.011_2 = 1 \times 2^2 + 1 \times 2^0 + 1 \times 2^{-2} + 1 \times 2^{-3} = 5.375。$$

整数部分可以用除以 2 取余数法，小数部分可以用乘以 2 取整数部分法。例如 $0.375 \times 2 = 0.75$，先取 0；$0.75 \times 2 = 1.5$，取 1 并留下小数部分 0.5；$0.5 \times 2 = 1$，再取 1。因此 $0.375 = 0.011_2$。

有限长度的二进制小数只能精确表示分母约分后是 2 的幂的有理数。例如 $3/8$ 可以有限表示，$1/10$ 不可以。后者会出现循环位，存储空间有限时就必须舍入。不要把“十进制写得短”误认为“二进制也写得短”。

浮点数把小数点的位置交给一个指数记录。比如 $101.011_2 = 1.01011_2 \times 2^2$。向左移动小数点两位，就要乘 2^2 补回来。它与十进制科学计数法相似：有效数字决定精细程度，指数决定大致大小。

来源：课程第 3 讲 PDF 第 3—6、10—11 页；教材 2.4.1—2.4.2。

2. 存下来的 exp、frac，不等于公式里的 E、M

先统一符号。一个格式有 1 个符号位、k 个阶码位、n 个小数字段位；总长度为 $1 + k + n$。以下写 `exp`、`frac` 时指**字段的位串**，写 e、F 时指这两个字段按无符号数解释的整数。

“字段”就是完整位串中划出来的一段。例如一个8位教学格式规定 $k = 3, n = 4$，存储内容是 `0 | 101 | 0110`：符号占1位，`exp` 是3位的 `101`，按无符号整数读得 $e = 5$；`frac` 是4位的 `0110`，按无符号整数读得 $F = 6$。这里 k, n 是长度，e, F 是字段的整数值；它们不是额外存储的数据。小数部分要再算 $F/2^n = 6/16$，不能直接把 $F = 6$ 当成有效数 M。

初学补充：Q014：字段、位串、k/n、e/F分别是什么意思？ 用同一组8位数据区分存储内容、字段整数值和公式中的E、M。

图 D2–1　浮点位布局与解释路径

从字段到数值：先分类，再解释（1/3）

1 · 先读位串，不能直接代入数值公式

s	exp	frac
1 位符号	k 位 → 无符号整数 e	n 位 → 无符号整数 F

最高位在左　　最低位在右

exp 非全 0、非全 1	**规格化**
exp 全 0	**非规格化或零**
exp 全 1	**无穷或 NaN**

exp 与 frac 是存储字段；E 与 M 是按类别解释出的指数和有效数。

从字段到数值：先分类，再解释（2/3）

2 · 规格化：恢复隐藏的前导 1

条件：$1 \leqslant e \leqslant 2^k - 2$

阶码字段 → 真实指数

$$E = e - Bias$$

$Bias = 2^{k-1} - 1$
exp 用偏置编码，不是补码。

小数字段 → 有效数

$$M = 1 + F / 2^n$$

前导 1 不占 frac 存储位。
$M = 1.frac_2$

$$V = (-1)^s \times M \times 2^E$$

从字段到数值：先分类，再解释（3/3）

3 · 非规格化：固定指数，取消隐藏的 1

条件：$e = 0$，$F \neq 0$

真实指数

$$E = 1 - Bias$$

与最小规格化指数相同，
让零附近的刻度平滑衔接。

有效数

$$M = F / 2^n$$

这里是 $0.frac_2$，
不能补上规格化的前导 1。

e=0，F=0 → ±0　　**exp 全 1，F=0** → ±∞　　**exp 全 1，F≠0** → NaN

符号位最后决定正负。先确认 exp 的类别，才能选择正确的 E 与 M；无穷和 NaN 不套有限数公式。

图里最关键的是“先分类”。不能看到阶码就机械地减偏置，因为全 0 与全 1 的阶码是特殊情况。对于普通规格化数，M 还包含一个没有存进去的前导 1。

若还不清楚图在做什么：它是在把存储位还原成“正负号 × 有效数 × 2的指数次方”。规格化数把frac接在 `1.` 后面；非规格化数把frac接在 `0.` 后面，并固定使用最小规格化指数，以表示更靠近零的数。两者是按阶码选择的不同规则，不是先后执行的两步。Q014追问：这两张规格化与非规格化图到底在干什么？ 用8位格式完整计算

5.5和0.09375，并说明为什么要减偏置、补前导1或改用前导0。

偏置是 $\mathrm{Bias}=2^{k-1}-1$。例如 $k=4$ 时 Bias=7。存下的 $e=5$ 对应真实指数 $E=5-7=-2$；负指数并没有用补码保存。exp 本身是按无符号数读出的，随后才减偏置。

类型	字段条件	指数 E	有效数 M 或含义
规格化数	$1\le e\le 2^k-2$	$e-\mathrm{Bias}$	$1+F/2^n$，即 $1.\mathrm{frac}_2$
非规格化数	$e=0$，$F\neq 0$	$1-\mathrm{Bias}$	$F/2^n$，即 $0.\mathrm{frac}_2$
正零、负零	$e=0$，$F=0$	可沿用非规格化公式	$s=0$ 是 +0，$s=1$ 是 −0
正无穷、负无穷	$e=2^k-1$，$F=0$	不套有限数公式	符号由 s 决定
NaN	$e=2^k-1$，$F\neq 0$	不套有限数公式	非数；不是某个特别大的实数

为什么非规格化的指数是 $1-\mathrm{Bias}$，不是 $0-\mathrm{Bias}$？因为从最大的非规格化数走到最小的规格化数时，希望步长平滑衔接。这里既取消隐藏的 1，又保留最小规格化指数，使靠近零的数按等间距排列。代价是越靠近零，有效数字可能越少。

常见格式可以从总位数检查记忆是否正确：单精度 binary32 为 1+8+23=32，Bias=127；双精度 binary64 为 1+11+52=64，Bias=1023。规格化数的有效精度分别是 **24 位和 53 位**，因为还包含隐藏的 1。不要把“23 位 frac”说成“只有 23 位有效数字”。本讲默认课程使用的 IEEE 二进制格式与最近偶数舍入。

23和52首先来自格式规定的位数分配：binary32规定符号占1位、阶码占8位，所以frac剩下32−1−8=23位；binary64规定符号占1位、阶码占11位，所以frac剩下64−1−11=52位。只知道总长度并不能唯一推出字段长度。

随后才有24=1+23、53=1+52：数的是有效数 $1.\mathrm{frac}_2$ 中的二进制位，前导1虽不存储，仍计入有效精度；不包含符号位和阶码位。Q014追问：23/52位frac和24/53位有效精度分别怎么来的？ 区分格式规定的分配与隐藏位带来的有效精度。

来源：课程第 3 讲 PDF 第 11—22 页；教材 2.4.2—2.4.3；第 3 讲普通研讨题 1(3)—(7)。

3. 解码和编码都用固定步骤

解码时，先把十六进制完整展开为固定位数的二进制，按 $1/k/n$ 分段；再分类、求 E,M，最后才计算符号和数值。十六进制的一位是 4 个二进制位，但字段分界不一定落在十六进制数字边界上。

编码时，先分离正负并把绝对值写成二进制。如果它处于规格化范围，就化成 $1.xxx_2\times 2^E$，再算 $e=E+\mathrm{Bias}$；如果太小，就使用非规格化固定步长；如果位数太多，要舍入。**最后检查舍入是否导致进位、类型变化或溢出。**

例题 D2-E01：把往年题拆成可重复的动作

来源：2022 期中无答案版（来源资料未随仓库分发），第二题第 1 小题、PDF 第 8 页。题意截取：一个 8 位格式的符号、阶码、小数字段长度是 1、3、4；解码 0xBD，再编码 $9/64$。按该题的“类似 IEEE”前提使用上述特殊字段规则。

第一问：0xBD 是多少?

1. 展开并分段：`10111101 = 1 | 011 | 1101`。
2. $k=3$，所以 Bias=$2^2-1=3$。$e=3$，既非全 0 也非全 1，是规格化数。
3. $E=3-3=0$。$M=1.1101_2=1+1/2+1/4+1/16=29/16$。
4. 符号位为 1，所以 $V=-29/16=-1.8125$。

第二问：$9/64$ 怎么存?

此格式最小规格化正数是 $2^{1-3}=1/4$。$9/64<1/4$，不能硬塞进规格化公式。非规格化数的步长为 $2^{1-3-4}=1/64$，于是 $9/64$ 正好是 9 个步长，$F=9=1001_2$。符号为 0，阶码全 0，拼成 `0 | 000 | 1001 = 00001001`，即 `0x09`。

再检查：代回公式 $(9/16)\times 2^{-2}=9/64$，正好一致。编码完成后代回解码公式，是很便宜也很有效的自检。

4. 范围和精度：浮点数是一把间距会变的尺

增加阶码位，主要扩大数量级跨度；增加 frac 位，主要在已有数量级之间放入更多刻度。固定总位数时，两者会发生取舍，所以“位数一样多”不能推出“范围一样”或“精度一样”。

对一个 $1/k/n$ 格式，有以下几个常用边界：

问什么	公式	怎样得到
最小正非规格化数	$2^{1-\text{Bias}-n}$	exp 全 0，frac 只有最低位为 1
最大正非规格化数	$(1-2^{-n})2^{1-\text{Bias}}$	exp 全 0，frac 全 1
最小正规格化数	$2^{1-\text{Bias}}$	exp 的整数值为 1，frac 全 0
最大有限正数	$(2-2^{-n})2^{2^k-2-\text{Bias}}$	exp 是全 1 减 1，frac 全 1
同一规格化指数区间内的间距	2^{E-n}	frac 增加 1，数值增加 $2^{-n}2^E$

这里“最小正数”绝不包括 0；“最小负数”按数值大小通常指最负的那个。若题目说“最小负非规格化数”，必须看它是指最负的非规格化数，还是绝对值最小的负数，不能擅自混用。

图 D2–2　一个 7 位格式在零附近的刻度（助手补充）

设 $k=3, n=3$，Bias=3。以下只画非负部分：



这幅图说明两个事实：非规格化与规格化交界处没有突然留下一个大洞；到新的二进制数量级之后，刻度间距会加倍。负数部分按符号镜像分布，但 +0 和 −0 的位模式不同、数值比较相等。

单精度在 $[2^{23}, 2^{24})$ 中的间距为 1，在 $[2^{24}, 2^{25})$ 中的间距为 2。因此从 0 到 2^{24} 的整数都能精确表示，但 $2^{24}+1$ 不行，$2^{24}+2$ 又可以。**不能精确表示一个整数，不意味着更大的整数都不能表示。**

一般地，若指数范围足够，n 位 frac 的格式能连续精确表示到整数 2^{n+1}；第一个不能精确表示的正整数是 $2^{n+1}+1$。题目若问“能精确表示的最大整数”，则是另一个问题，需要看最大有限数。例如 D2–E01 的格式最大有限数只有 15.5，其最大可精确表示整数为 15，不能盲套连续整数公式。答题时先圈出“最大”“连续”“所有”这些词。

来源：课程第 3 讲 PDF 第 25—29、48 页；教材 2.4.3、练习题 2.49。图 D2–2 为助手重绘。

5. 舍入：先找相邻刻度，再判断中点

默认规则的完整名字是“舍入到最近值，中点时取偶数”。大多数时候只是选最近的刻度；只有恰好在中间，才检查保留部分的最低有效位。二进制的“偶数”是这个位为 0，**不是要求最终十进制数为偶整数**。

你可以用两种等价方法。第一种是在数轴上找上下两个相邻可表示数，比较距离；第二种是在二进制上保留 n 位小数，检查删掉的部分：

删掉部分	与半个间距比较	动作
`0xxxx...`	小于一半	保留部分不变
`1000...`	恰好一半	保留部分最低位为 0 则不变，为 1 则加 1
`1xxxx...` 且后面还有 1	大于一半	保留部分加 1

这里“加 1”是给保留部分的末位加一个单位，可能连续进位。负数同样比较距离；遇到中点，也让保留后的最低有效位是 0。不要把“负数都向零靠”当成最近偶数舍入，那是另一种模式。

Q015追问：D2–E03的最近偶数判断，就是表里的1000…这一行吗？ 将精确结果切成保留的23个小数位和舍弃的 `1000…`，展示连续进位及恢复负号的过程。

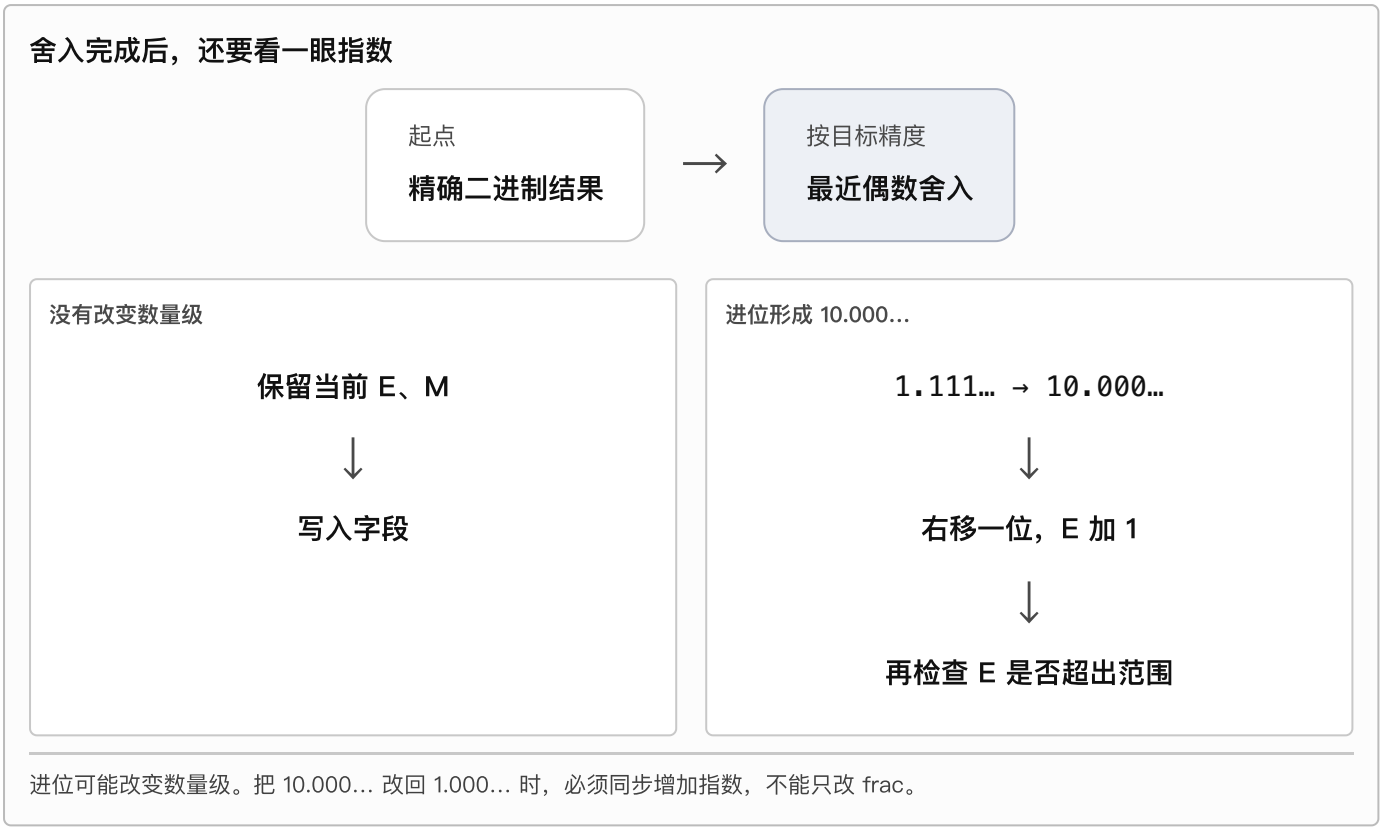
例题 D2–E02：中点舍入为什么还要改指数?

助手补充，格式为 8 位 `1/4/3`，Bias=7。把 13.5 和 15.5 编码，均用最近偶数舍入。

对 $13.5 = 1101.1_2 = 1.1011_2 \times 2^3$，只能保留 `1.101`。删掉的是 `1000...`，恰好在 13 和 14 中间；保留末位是 1，必须进位到 `1.110`。指数仍为 3，存储阶码为 10，即 `1010`，得到 `0 | 1010 | 110 = 0x56`，数值为 14。

对 $15.5 = 1111.1_2 = 1.1111_2 \times 2^3$，仍是中点，末位 1 要进位。可是 `1.111 + 0.001 = 10.000`，已经不符合规格化的 `1.xxx` 形式。要改写为 1.000×2^4。因此新 $e = 4+7 = 11$，得到 `0 | 1011 | 000 = 0x58`，数值为 16。

图 D2–3　舍入后的检查流程



图中最后一次检查不能省。如果重新规格化后已经没有可用有限阶码，在最近偶数模式下可能得到无穷。很小的数也会在舍入后变成零，或从非规格化跨入最小规格化数。并不是所有大于最大有限数的实数都立刻舍成无穷；靠近上边界时仍应按目标格式的舍入规则判断。

若看到课件中的 G、R、S，不必把它当成另一套原理。第 3 讲第 46 页把 **G 定为最后保留的位，R 定为第一位被删的位，S 定为后续所有被删位的按位或**。在这一命名下，进位条件是 $R \wedge (S \vee G)$：R=0 不到一半；R=1 且 S=1 超过一半；R=1、S=0 恰好一半，此时由 G 决定。其他材料可能把 G 命名在第一位被删的位置，做题一定先读定义，别只记三个字母。

材料校验：第 3 讲 PDF 第 47 页有数值笔误，例如 $\texttt{1.001}_2 \times 2^7$ 写成 134，实际是 144。本讲重新计算所用例题，不照抄该页结果。

来源：课程第 3 讲 PDF 第 31—36、44—47 页；教材 2.4.4—2.4.5；第 3 讲普通研讨题 2(1)—(4)。

6. 浮点加乘与 C 转换：每次运算都可能丢信息

本阶段把浮点运算理解为“先求精确结果，再舍入到目标格式”就够了。加法先对齐二进制小数点，再按符号加减、规格化和舍入；乘法先异或符号位、相乘有效数、相加指数，再规格化和舍入。乘法的指数是**真实指数 E 相加**，不能直接把两个带偏置的 exp 字段相加当作答案。

例如在 `1/4/3` 格式中，$1.5 \times 1.75 = 2.625 = 1.0101_2 \times 2^1$，位于 2.5 和 2.75 中间。保留 `1.010` 的末位为 0，所以存成 2.5。算浮点乘法时只写精确积 2.625，会漏掉最后一层舍入。

例题 D2–E03：为什么不能任意移动括号?

助手补充。假设每一步都按 binary32 运算并舍入，令 $a = 2^{25}$、$b = -2^{25}$、$c = 1$。

第一种括号 $(a + b) + c$：先精确抵消得 0，再加 1，结果为 1。第二种括号 $a + (b + c)$：先算 $-2^{25} + 1$，它在相邻可表示值 -2^{25} 与 $-2^{25} + 2$ 的中间，最近偶数规则选择 -2^{25}，再与 a 相加得 0。

展开看：精确结果是−33,554,431，左右可表示值是−33,554,432和−33,554,430，各差1。中点取偶数看的是有效数最低保留位，不是十进制整数奇偶；前一候选的frac全0，后一候选的frac全1，所以选前者。Q015：为什么-2^{25}加1会先舍回-2^{25}，最后得到0？ 按“精确求和→找相邻值→判断末位→再加a”逐步展开。

两种写法的实数结果相同，浮点结果却不同。反例的要点是每一步明确舍入，不能仅说“大数吞小数”。特别要注意二次幂附近的刻度：某个数左右两边的间距可能不同。找中点时应把真正相邻的两个可表示数写出来，不要看见量级很大就断言加 1 一定消失。

浮点加法、乘法一般不满足结合律，乘法对加法一般也不满足分配律。对于有限且不产生 NaN 的相应运算，非严格的单调关系仍有用；严格大于可能在舍入后变成相等。遇到无穷和 NaN，要另查特殊规则，不能把实数代数直接搬过来。

以下转换规则默认 `int` 是 32 位，`float/double` 分别为 binary32/binary64：

这里“转换”是把一个数值从一种数据类型转换成另一种类型。例如 `int x=3; float y=x;` 会读取整数3，把它转换成float的3.0并存入y，x仍是int且值不变；`int → float` 只是转换方向的写法，不是C运算符。表格关心的是转换后能否保留原值，还是必须舍入、去掉小数部分或遇到范围限制。Q016：这张表里的类型转换是什么意思？ 用实际C代码说明自动转换、显式转换及与原位串重新解释的区别。

转换	应当先想到什么
`int → float`	范围足够，但 24 位精度可能不够，可能舍入
`int → double`	53 位精度足以精确表示所有 32 位 int
有限 `float → double`	原来的数值可以精确保留，不会补回以前已经丢掉的信息
`double → float`	可能舍入，也可能超出 float 范围；课程 IEEE 模型下可能得到无穷
有限 `float/double → int`	去掉小数部分，向零截断；还要检查截断结果能否由 int 表示

例如 `(int)-3.9` 是 −3，不是 −4；这与浮点算术默认的最近偶数舍入不是一回事。NaN、无穷或截断后超出整数范围的浮点到整数转换，不能当成可移植 C 的正常数值运算；不要无条件回答“必为 TMin”。具体指令可能产生某种硬件结果，那需要题面提供机器前提。

还要检查表达式的类型：`2/3` 先做整数除法，结果为 0；`2/3.0` 做浮点除法。赋值给 `double` 并不能倒回去把已经完成的整数除法重新算一遍。

7. 零、无穷、NaN 与“看位”

在课程采用的 IEEE 语义下，+0 与 −0 的位不同，但 `+0.0 == -0.0` 为真；其符号在倒数等运算中仍可观察，例如正数除以 +0 与 −0 会给出不同符号的无穷。NaN 更特殊：`NaN == NaN` 为假，`NaN != NaN` 为真，与 NaN 做 <、<=、>、>= 都为假。不能用 `x == NaN` 判断是不是 NaN，应按题意使用 `isnan`，或在明确 IEEE 前提时利用 `x != x`。

“转换成整数”与“把相同位串看成整数”是两件不同的事。前者尽量保留数值，改变编码；后者保留编码，改变解释。比如 binary32 的 1.0 编码是 `0x3f800000`，数值转换成 int 得 1；把其位串读作 32 位整数则得到 1065353216。后者不是 `(int)1.0f` 的含义。如何在 C 中安全操作这些字节，会在联合与内存一讲继续解释。

例题 D2-E04：往年题中的“半字节浮点”

来源：2024 期末无答案版（来源资料未随仓库分发），第一题第 1 小题、PDF 第 2 页。按题意截取并改为表格分析：`float4` 有 1 个符号位、2 个阶码位、1 个 frac 位；`int4` 是 4 位补码。`f2b` 保留位串后按 int4 解释；`f2i` 把浮点值向零转换为 int4。原题用假想类型说明规则，不是能直接编译的标准 C 类型。

判断候选位串 `0000`、`0001`、`1000`、`1101` 中，哪些使二者相等。

先读懂任务：每个候选给的是原始4位，不是已经确定的整数值。对同一个候选，分别走两条独立路线：**f2b直接把原位串按4位补码读；f2i先按float4解出浮点值，再向零取整。** 比较的是两条路线最后得到的整数，不是让一个函数接着另一个函数执行。例如 `0001` 直接按补码读是1，先按浮点读却是0.5，再取整得到0，所以不相等。Q017：半字节浮点题到底给了什么、要比较什么？ 逐项解释两种类型、两条路线和四个候选。

Bias=$2^1-1=1$。逐个按 `s|exp|frac` 分段：

位串	浮点值	同位串的 int4 值，即 f2b	向零转换的 f2i	是否相等
`0|00|0`	+0	0	0	是
`0|00|1`	$0.1_2\times 2^0=1/2$	1	0	否
`1|00|0`	−0	−8	0	否
`1|10|1`	$-1.1_2\times 2^1=-3$	−3	−3	是

故相等的是 `0000`、`1101`，对应原题选项 D。最容易错的是把 −0 的位串当成整数的 0，以及把 `0001` 在两种格式下都解释成 1。题目在考解释规则，不能只看位串长得像哪个熟悉的数字。

8. 条件码：CPU 记下的四个一位结果

汇编没有直接写出的 `if (a<b)`。常见实现是先计算一次比较、保存几个标志，再让跳转或赋值指令查看标志。四个标志是：

标志	问的问题	初学时的检查方法
ZF	固定位宽结果是否全 0?	全 0 为 1，否则 0
SF	结果最高位是否为 1?	直接抄结果最高位
CF	无符号加法是否进位 / 减法是否借位?	加法超出无符号范围；减法时无符号被减数小于减数
OF	补码意义下是否溢出?	加法同号输入变异号；减法异号输入且结果符号异于被减数

先用4位教学运算理解：无符号范围是0～15，补码范围是−8～7。`0111+0001=1000` 按无符号读是7+1=8，没有向第5位进位，CF=0；按补码读却把本应为8的结果存成−8，超出有符号范围，OF=1。对于减法A−B，A叫被减数、B叫减数；CF检查无符号A是否小于B，OF检查补码意义下的精确差是否越界。Q018：CF的进位/借位与OF的有符号溢出是什么意思？ 逐位展开加法与减法，并解释表中的符号判断。

ZF、SF 只需看截断后的结果。CF、OF 要结合运算与输入；它们互不等价。一次运算可以只有 CF=1、只有 OF=1、两者都为 1，或都为 0。SF=1 也不能说明“算错了”或“发生溢出了”，它只描述结果位串的最高位。

AT&T 语法遵循“源在前、目的在后”。因此 `subq b,a` 做 $a\leftarrow a-b$；`cmpq b,a` 也按 **a−b** 设置标志，但不把差写回 a。记成“比较后一个与前一个”，然后在草稿上写出差，就不容易反过来。

`testq b,a` 根据 `a & b` 设置标志，也不改写操作数。它把 CF、OF 清零，ZF、SF 看按位与结果。`testq %rax,%rax` 实际检查 rax 是否为零、符号位是否为 1；`testb $1,%al` 检查最低位是否为 1，常用于奇偶判断。前者不是真的计算平方，后者也不是真的比较 al 与 1。

图 D2–4　比较、标志与三类使用方式

先用“是否相等”读这张图。假设rax和rbx都为5，执行 `cmpq %rbx, %rax`，CPU按5−5的结果设置标志：ZF=1，但rax、rbx仍为5。后面的指令不重新比较这两个寄存器，而是查看已保存的ZF。图中的 `cc` 是“条件后缀”的占位符，例如换成 `e` 表示相等，此时检查ZF=1。



下面三行分别说明图中的三种用法，每行都独立接在上述比较后理解，不要求按顺序执行：

实际指令	条件成立（ZF=1）时	条件不成立（ZF=0）时
`sete %cl`	把cl这个字节写成1	把cl写成0
`je .L_equal`	跳到标签.L_equal标出的代码位置	继续执行紧接着的下一条指令
`cmove %rsi, %rdx`	把rsi的值复制到rdx	rdx保持原值

因此setcc是把判断结果存成0或1，jcc是选择接下来执行的位置，cmovcc是有条件地复制数据。标签只是代码位置的名字；“写一个字节”也不是“只写一个位”。Q019：cmp之后的setcc、jcc、cmovcc到底分别在做什么？从5与5的比较出发，并对照mov保留标志、add覆盖标志的过程。

读图时注意，条件指令读取的是**所需标志当前保存的值**，要追踪每个相关标志最近由谁修改，不是固定读取上方最近的cmp。`mov`、`lea`、`setcc`、`jcc`、`cmovcc` 不会重新设置这四个标志；`add/sub` 会，逻辑运算会。`inc/dec` 会修改ZF、SF、OF，但保留CF。例如比较相等后ZF=1，中间的mov不会改变它；若再执行一次结果非零的add，ZF就改成0，随后je不再跳转。

不要把移位指令笼统记成“OF 一律清零”。真实 x86 的移位标志与指令及有效移位次数有关：次数为 0 时不改标志；单个位移的 OF 规则依指令而异，多位移的 OF 通常未定义。本讲练习不靠未定义的 OF 出题。教材 3.6.1 对移位 OF 的一句简化表述不能当作所有移位的通用规则。

来源：课程第 5 讲 PDF 第 13—25 页；教材 3.6.1—3.6.2；第 5 讲普通研讨题 1(1)—(5)。

9. 同一次 cmp，为什么有符号和无符号会走不同分支？

`cmp` 自己没有 signed 或 unsigned 版本：同样宽度的减法产生同一组标志。区别在后面选择哪个条件。下表按刚刚执行的 `cmp b,a` 来读，“a<b”中的 a 是被减数。

关系	signed 后缀	条件	unsigned 后缀	条件
a=b	e 或 z	ZF	同左	同左
a≠b	ne 或 nz	非 ZF	同左	同左
a<b	l	SF≠OF	b	CF
a≤b	le	(SF≠OF) 或 ZF	be	CF 或 ZF
a>b	g	(SF=OF) 且非 ZF	a	非 CF 且非 ZF
a≥b	ge	SF=OF	ae	非 CF

后缀可以接在 `set`、`j`、`cmov` 后面。例如 `setl %al` 把有符号小于关系写成 0/1，`jl .Lsmall` 在同一条件下跳转。这里 `setl` 的 l 是 less，不是 32 位 long 后缀；所有 setcc 都只写一个字节。

这些是本讲需要会读的常用指令，先按“动作＋条件”拆开，不把每种组合当成全新的命令背。`set+l` 是“满足有符号小于条件就写1，否则写0”；`j+l` 是“满足同一条件就跳转，否则继续下一条”；`cmov+l` 是“满足同一条件就复制，否则保留目的原值”。例如rdi=3、rsi=5，先执行 `cmpq %rsi,%rdi`，再执行 `setl %al` 得到al=1；若改接 `jl .Lsmall`，则跳到标签.Lsmall。条件指令只读取已有标志，并不重新比较两个操作数。

注意区分指令类别：`movl/addl/cmpl` 的l表示32位宽度；`setl/jl/cmovl` 的l表示有符号less条件。`setl %al` 只写8位al，不会把整个rax都写成0或1。第10节再解释怎样整理高位。Q019追问：这些指令需要掌握吗，setl和jl该怎样拆开读？ 给出本讲练习所需的学习范围与条件后缀速查，不代表完整考试指令清单。

为什么有符号小于使用 SF 异或 OF，而不直接看 SF？没有溢出时，差为负当然意味着 a<b；有溢出时，补码结果的符号反了，需要 OF 把判断再翻一次。CF 则保留无符号减法是否需要借位的信息，所以无符号比较只需另选对应条件。

记忆时先抓三条：相等看ZF=1，无符号小于看CF=1，有符号小于看SF≠OF。再由“小于等于＝小于或相等”“大于等于＝不是小于”“大于＝不是小于且不相等”推其余条件。Q019追问：条件表可以直接背吗，怎样少记几条？说明推导方法与比较顺序、标志未被覆盖的前提。

例题 D2-E05：SF 为 1，也可能是 a>b

助手补充。8 位寄存器 al 初值是 `0x64`，执行 `cmpb $0xd8,%al`。

先按无符号整数读操作数：`0x64=100`、`0xd8=216`，精确差是−116。只保留低8位，相当于取模256：−116+256=140，而140=8×16+12，所以结果位串是 `0x8c`，即 `1000 1100`。因此这里的 `0x64-0xd8 → 0x8c` 表示固定8位运算的结果，不是普通整数等式。al仍是 `0x64`，因为cmp不写回。Q020：0x64减0xd8，为什么低8位是0x8c？ 用十进制换算、十六进制借位及两种数值解释逐步核对。

1. 差非零，ZF=0；最高位为 1，SF=1。
2. 按无符号解释，100−216 需要借位，CF=1。
3. 按补码解释，`0xd8` 是 −40，真实差是 100−(−40)=140，超过 8 位补码最大值 127，OF=1。
4. signed 小于条件是 SF≠OF，这里不成立；signed 大于条件成立。因此 `jg` 会跳，符合 100>−40。
5. unsigned 小于条件是 CF=1，所以 `jb` 会跳，符合 100<216。

同一条 cmp、同一组标志，两种判断都正确，因为问的是两种不同解释下的关系。

10. setcc 写一字节，返回整数还需整理高位

考虑返回 `x < y` 的函数，其中 x、y 是 64 位有符号 long，分别在 rdi、rsi：

```
cmpq    %rsi, %rdi       # 比较 x 与 y：概念差 x-y
setl    %al              # 只把 al 写成 0 或 1
movzbl  %al, %eax        # 字节零扩展到 32 位；写 eax 又清零 rax 高32位
ret
```

假设原 rax 是 `0xffffffffffffffff`，比较结果为假。只执行 `setl %al` 后，rax 是 `0xffffffffffffff00`，还不是整数 0。执行 `movzbl` 后才成为全 0。比较结果为真时，则整理成完整的整数 1。

`movzbl` 名字中的 b 和 l 表示从 8 位扩展到 32 位，不是两个不同的条件。写 `%al` 不会清除其余位；写 `%eax` 会自动清除 `%rax` 高 32 位。也不要在 `cmp` 与 `setcc` 之间随手插入 `xor %eax,%eax` 清零，它会改写条件码，破坏原比较结果。若要先清零，应放在比较之前。

11. if 与 cmov：先说明执行了什么，再谈结果

例题 D2-E06：把有符号最大值写成两种实现

助手补充。函数返回两个 long 中较大者，不涉及加减溢出：

```
long larger(long x, long y) {
    if (x > y) return x;
    return y;
}
```

一份便于阅读的分支实现是：

```
cmpq    %rsi, %rdi       # x-y；检查 x>y
jg      .Lx
movq    %rsi, %rax       # 未跳：x<=y，取 y
ret
.Lx:
movq    %rdi, %rax       # 已跳：x>y，取 x
ret
```

取 x=−2、y=3。比较后 `jg` 不满足，程序顺序执行 `movq %rsi,%rax`，返回 3；它不会又走到 `.Lx`，因为已经 ret。读分支题时，应沿一条真实路径走，不要把两个分支的赋值都叠加起来。

同一函数也可写为：

```
movq    %rsi, %rax       # 先准备候选 y
cmpq    %rsi, %rdi
cmovg   %rdi, %rax       # 若 x>y，用 x 替换；否则保持 y
ret
```

`cmovg` 不跳转，它只决定复制不复制。两份代码都返回最大值，但这不意味着任意 if 都可以把两边提前计算。以下是普通研讨题要求理解的三类限制：

- **可能出错的表达式。** `p ? *p : 0` 在 p 为空时本来不访问 `*p`。若先加载 `*p` 再用 cmov 选 0，访问错误已经发生；结果最终选 0 也救不了。
- **带副作用的表达式。** 只有选中的分支应修改变量、写内存或调用带副作用的函数，不能把两边都执行一遍再只挑结果。
- **计算代价很大的表达式。** 两边都计算可能浪费大量工作，即使语义允许，也未必更快。

特别地，条件传送的内存源操作数也不能被想象成“条件为假就一定不访存”。做安全判断，应先检查候选值的获取是否可能触发错误。上述空指针分支可以使用正常跳转；更复杂的安全改写需要另证正确性，本轮不为它扩展新技巧。

读短汇编时在每个分支前写四项：当前类型/位宽、比较的真实顺序、跳转条件、跳与不跳各到哪里。循环与 switch 会在下一讲建立在这个流程上。

来源：课程第 5 讲 PDF 第 27—35 页；教材 3.6.3、3.6.5—3.6.6；第 5 讲普通研讨题 1(6)—(7)、1(9)。不布置其中的拓展研讨。

12. 独立练习：先交过程，再核对答案

以下全部为助手补充的独立训练题，编号固定为 D2-P01—D2-P10。必做 8 题共 30 分：4 道浮点短题、3 道条件短题、1 道短分支对照题。它们不是额外的一套整卷，也不是用户已完成的作答记录。

默认 x86-64、Linux、AT&T 语法；所有浮点题用题面给定的 IEEE 式字段规则和最近偶数舍入；不讨论信号 NaN 的异常行为。写出字段、公式或比较关系，仅写最后一个数不足以定位错误。

D2-P01　解码两个不同类别的数（4 分）

8 位浮点格式为 `1/4/3`。分别解码 `0x25` 和 `0x83`。每个数写出分段、类别、E, M 和精确数值，数值可写成分数。

D2-P02　范围与特殊值（4 分）

7 位格式为 `1/3/3`。求最小正非规格化数、最小正规格化数、最大有限正数；再判断 `0 | 111 | 010` 表示什么。每项 1 分，边界题必须写出依据。

D2-P03　中点与重新规格化（4 分）

将 $21/4$ 与 $31/8$ 编码成 `1/4/3` 的 8 位浮点数。每问写出精确二进制形式、舍入方向与理由、舍入后数值和十六进制编码。

D2-P04　数值转换（3 分）

假设 int 为 32 位，float/double 为 binary32/binary64，使用默认最近偶数舍入。

1. 对所有 int `x`，`x == (int)(double)x` 是否成立？说明原因。
2. 若 `int x = 16777219;`，求 `(int)(float)x`，说明邻近刻度与舍入理由。
3. 求 `(int)-6.75`，说明使用哪一种取整方向。

第2小题追问：Q021：16777219在两个刻度中间，为什么选16777220？（含该小题解答；展开最低保留位、刻度单位与中点进位，按需打开）。

D2-P05　分别计算 CF 和 OF（4 分）

下面两段独立执行。分别给出最后的 al 与四个标志 CF、ZF、SF、OF。

```
# A
movb $0x7f, %al
addb $1, %al

# B
movb $0x00, %al
subb $1, %al
```

D2-P06　选择 signed 与 unsigned 条件（3 分）

设 edi=`0xffffffff`，esi=`0x00000001`。执行 `cmpl %esi,%edi` 后，分别求 `setl %al` 与 `setb %al` 会写入的值，并说明二者为何不同。两条 set 指令均依据同一次 cmp 的标志，不要把其中一次 set 当成新的比较。

D2-P07　部分寄存器与条件码保持（3 分）

初始 rax=`0xaabbccddeeff0011`，edi=0。依次执行：

```
testl   %edi, %edi
setne   %al
movzbl  %al, %eax
```

写出执行 setne 后的完整 rax、执行 movzbl 后的完整 rax，以及最终 CF、ZF、SF、OF。说明为什么不能把 setne 后的 rax 直接当作布尔返回值。

D2-P08　短分支的 C 与汇编对照（5 分）

参数 x、y 是 signed long，分别位于 rdi、rsi；假设本题实际执行的加减结果都在 long 范围内。

```
choose:
    cmpq %rsi, %rdi
    jg   .Lelse
    leaq 2(%rdi), %rax
    ret
.Lelse:
    leaq -1(%rsi), %rax
    ret
```

1. 补全 C：`if (_____) return x+2; else return y-1;`。
2. 分别计算输入 `(x,y)=(-3,2)` 与 `(7,2)` 的返回值。
3. 若改为 unsigned long 比较、仍保持同样的“小于等于时走第一分支”意图，`jg` 应换成什么？
4. 用一句具体理由说明：能否把任意 `p ? *p : 0` 都改成“先读 `*p`，再条件传送”？

D2-P09　选做：同样位数，为什么有限实数数量不同？

两个 7 位格式，A 为 `1/3/3`，B 为 `1/4/2`。分别求它们表示的**不同有限实数**的数量，+0 与 −0 只算一个，NaN 和无穷不算。然后比较最小正非规格化数。不要只根据“B 指数多一位”猜数量；列出允许的 exp、frac、符号组合。

D2-P10　选做：更大不代表转换后仍严格更大

令 int a=16777216，int b=16777217。先比较 a、b；再求 binary32 中 `(float)a`、`(float)b`，判断它们之间的 `<` 和 `==`。若先分别转换成 double 再比较，结论如何？解释这是否反驳了舍入的**非严格**单调性。

13. 过关检查与资料定位

完成必做题后按答案中的作答点自评，24/30 是本讲首次独立练习的门槛。即使总分达标，若仍分不清 exp 与 E，或者把 `cmp b,a` 反读为 b−a，先修这两个前置错误再学循环。没达到门槛时，不重抄整章：把错误归为“分段/分类”“舍入”“操作数顺序”“条件类型”“部分寄存器”之一，每类重做一个更小例子，再做未见变式。这里不预填你的分数或掌握状态。

合上讲义，检查自己能否说清：非规格化为什么不用隐藏 1；最近偶数只在什么情况下看末位；浮点转 int 为何不使用相同舍入规则；为什么 cmp 不改寄存器却会影响跳转；为什么 SF=1 不能单独证明 a<b；为什么 setcc 后常跟 movzbl。能说清再做对题，才算把规则接到了操作上。

本讲使用的文件与精确定位如下。下列 PDF 页序从文件第一页开始；教材只引用已核对的小节，不把目录页序当印刷页码。

- 课程第 3 讲：浮点（来源资料未随仓库分发）：PDF 第 3—6、10—42、44—48 页。
- 课程第 5 讲：控制（来源资料未随仓库分发）：PDF 第 13—35 页；循环和 switch 留到第 3 讲复习。
- 教材《深入理解计算机系统》第 3 版（来源资料未随仓库分发）：2.4.1—2.4.6、3.6.1—3.6.3、3.6.5—3.6.6；部分寄存器规则参见 3.4.2。当前本地文件已核对 2.4.4—2.4.6 为印刷页 83—86、PDF 第 119—122 页，3.6.1—3.6.2 的相关说明为印刷页 135—137、PDF 第 171—173 页，3.6.6 为印刷页 145—148、PDF 第 181—184 页。
- 第 3 讲研讨题（来源资料未随仓库分发）：PDF 第 1—2 页的普通研讨要求；排除明确标为拓展的事故调研、NaN 载荷与阿贝尔群等题。
- 第 5 讲研讨题（来源资料未随仓库分发）：PDF 第 1 页“条件码和条件分支”普通题；本讲不要求流水线性能调研。

例题与独立题属于教学材料，后续你的真实提问和追问另按稳定编号记录，不把这些预先写成你已经提出的问题。